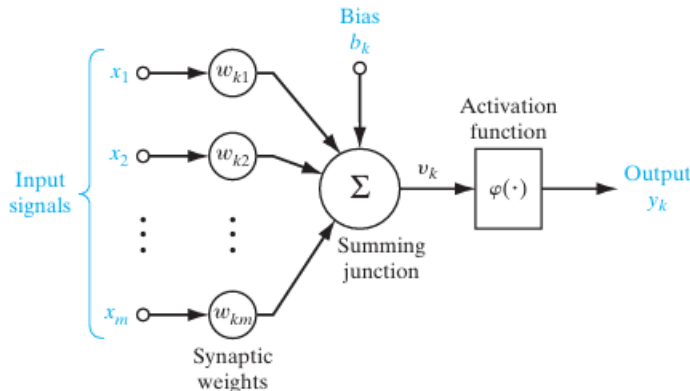
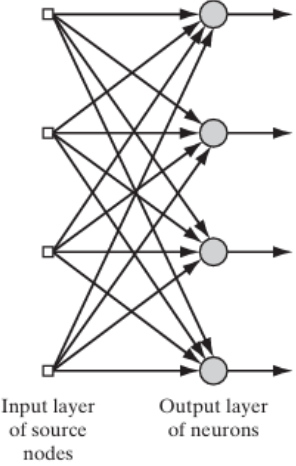
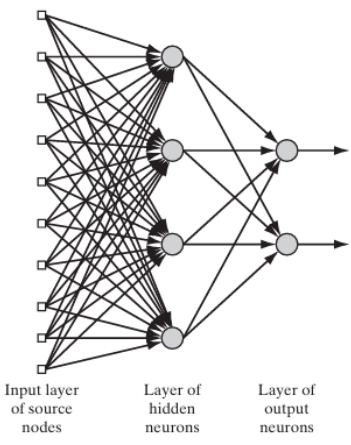
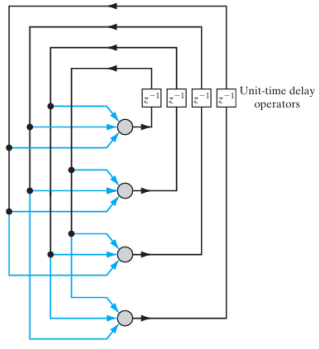


	RV College of Engineering® Department of Computer Science and Engineering CIE - I: Test Paper	
Course & Code	Deep Learning (CS374TFA)	Semester: VII
Scheme & Solutions		

Sl.no.	Part A- Questions	Marks															
1.	Convolution is an operation that combines two functions — an input function x and a kernel (filter) w — to produce a third function $s(t)$, which expresses how the shape of one is modified by the other. Mathematical Expression (Continuous form): $s(t) = (x * w)(t) = \int_{-\infty}^{\infty} x(a) w(t - a) da$ Discrete form: $s(t) = (x * w)(t) = \sum_{a=-\infty}^{\infty} x(a) w(t - a)$	2															
2.	It can only model linearly separable problems and cannot handle non-linear decision boundaries.	1															
3.	The convolution operation is useful for feature extraction because it applies the same local linear transformation across all spatial locations, allowing the network to detect local patterns such as edges or textures anywhere in the input. This weight sharing and local connectivity give the model translation invariance and make it highly efficient for visual feature learning.	2															
4.	We divide the input into 2×2 non-overlapping regions and take the mean of each: <table border="1" data-bbox="321 1465 1243 1732"> <thead> <tr> <th>Region</th><th>Values</th><th>Average</th></tr> </thead> <tbody> <tr> <td>(1)</td><td>[1, 3, 5, 6]</td><td>$(1+3+5+6)/4 = 3.75$</td></tr> <tr> <td>(2)</td><td>[2, 4, 1, 2]</td><td>$(2+4+1+2)/4 = 2.25$</td></tr> <tr> <td>(3)</td><td>[3, 1, 1, 2]</td><td>$(3+1+1+2)/4 = 1.75$</td></tr> <tr> <td>(4)</td><td>[2, 0, 3, 4]</td><td>$(2+0+3+4)/4 = 2.25$</td></tr> </tbody> </table> Pooled Output = $\begin{bmatrix} 3.75 & 2.25 \\ 1.75 & 2.25 \end{bmatrix}$	Region	Values	Average	(1)	[1, 3, 5, 6]	$(1+3+5+6)/4 = 3.75$	(2)	[2, 4, 1, 2]	$(2+4+1+2)/4 = 2.25$	(3)	[3, 1, 1, 2]	$(3+1+1+2)/4 = 1.75$	(4)	[2, 0, 3, 4]	$(2+0+3+4)/4 = 2.25$	2
Region	Values	Average															
(1)	[1, 3, 5, 6]	$(1+3+5+6)/4 = 3.75$															
(2)	[2, 4, 1, 2]	$(2+4+1+2)/4 = 2.25$															
(3)	[3, 1, 1, 2]	$(3+1+1+2)/4 = 1.75$															
(4)	[2, 0, 3, 4]	$(2+0+3+4)/4 = 2.25$															

5.	The two main characteristics of knowledge representation are: □ What information is made explicit — i.e., the specific facts or relationships that are represented. □ How the information is physically encoded — i.e., the form or structure in which the knowledge is stored for later use.	2
6.	Learning with a teacher (supervised learning) and without a teacher (which includes unsupervised and reinforcement learning).	1

Sl.no.	Part B- Questions	Marks
1.a	A neuron is an information-processing unit that is fundamental to the operation of a neural network. Important elements of a neuron include: 1. A set of synapses, or connecting links, each of which is characterized by a weight or strength of its own. Specifically, a signal x_j at the input of synapse j connected to neuron k is multiplied by the synaptic weight w_{kj}. It is important to make a note of the manner in which the subscripts of the synaptic weight w_{kj} are written. The first subscript in w_{kj} refers to the neuron in question, and the second subscript refers to the input end of the synapse to which the weight refers. Unlike the weight of a synapse in the brain, the synaptic weight of an artificial neuron may lie in a range that includes negative as well as positive values. 2. An adder for summing the input signals, weighted by the respective synaptic strengths of the neuron; the operations described here constitute a linear combiner. 3. An activation function for limiting the amplitude of the output of a neuron. The activation function is also referred to as a squashing function, in that it squashes (limits) the permissible amplitude range of the output signal to some finite value.  In mathematical term , we may depict neuron k as follows: $u_k = \sum_{j=1}^m w_{kj} x_j$	10

2.	i) Single-Layer Feedforward Networks ii) Multi-Layer Feedforward Networks iii) Recurrent Neural Networks  Input layer of source nodes Output layer of neurons Single Layer FFN  Input layer of source nodes Layer of hidden neurons Layer of output neurons Multi-Layer FFN  Recurrent Networks	10
3.	One of the key roles of pooling is to make the representation approximately invariant to small translations of the input. □ Translation invariance: Pooling helps ensure that small shifts or distortions in the input don't overly change the output of the layer — the network learns that a feature may appear slightly shifted but is still the same. □ Dimensionality reduction & computational efficiency: By pooling over regions, the size of the feature map shrinks, reducing the number of downstream computations and parameters required. □ Prior assumption encoded: The use of pooling encodes a domain-knowledge assumption (prior) that we care about <i>presence</i> of feature more than <i>exact location</i>. Goodfellow describes pooling as imposing “an infinitely strong prior” for invariance. □ Feature abstraction: Pooling layers help transition from fine-grained details (edges, small patterns) to coarser, more abstract representations (parts, objects) by summarizing over regions. Among the most common types are max pooling, average pooling, and L2 (energy) pooling. In max pooling, the maximum activation within a local region is selected, allowing the network to focus on the most prominent features—such as edges or corners—while providing strong local translation invariance, since a small shift in the feature location does not change the pooled value. Average pooling, on the other hand, computes the mean of activations within a region, producing smoother, more distributed feature maps that preserve contextual information but offer weaker invariance. L2 pooling computes the root-mean-square of activations, combining the advantages of both by capturing the overall energy of activations and offering a balance between selectivity and robustness.	10

4.	Let's compute the induced local field v first, then apply each activation. 1) Induced local field $v = \sum_{j=1}^3 w_j x_j + b$ $= (0.8)(0.5) + (-0.4)(-1.0) + (0.3)(2.0) + 0.2$ $= 0.40 + 0.40 + 0.60 + 0.20$ $= 1.60$ 2) Threshold (Heaviside) activation $y = \begin{cases} 1, & v \geq 0 \\ 0, & v < 0 \end{cases} \Rightarrow y = 1 \text{ (since } v = 1.6)$ 3) Sigmoid (logistic) activation $\phi(v) = \frac{1}{1 + e^{-v}}$ $e^{-1.6} \approx 0.2019, \quad \phi(1.6) = \frac{1}{1 + 0.2019} \approx \frac{1}{1.2019} \approx 0.8320$ Output: ≈ 0.8320 4) Hyperbolic tangent activation $\phi(v) = \tanh(v)$ $\tanh(1.6) = \frac{e^{1.6} - e^{-1.6}}{e^{1.6} + e^{-1.6}} = \frac{4.9530 - 0.2019}{4.9530 + 0.2019} = \frac{4.7511}{5.1549} \approx 0.9217$ Output: ≈ 0.9217 Summary of outputs (for $v = 1.6$) • Threshold: 1 • Sigmoid: ≈ 0.8320 • Tanh: ≈ 0.9217	10
5.	In convolution, the kernel is flipped both horizontally and vertically before sliding Flipped kernel: $K_{flip} = \begin{bmatrix} 1 & -1 \\ 0 & 1 \end{bmatrix}$ Now, slide this 2×2 kernel across I. The output will be of size 2×2 because: $(3 - 2 + 1) \times (3 - 2 + 1) = 2 \times 2$	10

Step-1:Top-left region of I :

$$\begin{bmatrix} 1 & 2 \\ 4 & 5 \end{bmatrix}$$

$$S_{11} = (1)(1) + (2)(-1) + (4)(0) + (5)(1) = 1 - 2 + 0 + 5 = \boxed{4}$$

Step-2:

Top-right region:

$$\begin{bmatrix} 2 & 0 \\ 5 & 1 \end{bmatrix}$$

$$S_{12} = (2)(1) + (0)(-1) + (5)(0) + (1)(1) = 2 + 0 + 0 + 1 = \boxed{3}$$

Step-3:

Bottom-left region:

$$\begin{bmatrix} 4 & 5 \\ 2 & 3 \end{bmatrix}$$

$$S_{21} = (4)(1) + (5)(-1) + (2)(0) + (3)(1) = 4 - 5 + 0 + 3 = \boxed{2}$$

Step-4:

Bottom-right region:

$$\begin{bmatrix} 5 & 1 \\ 3 & 2 \end{bmatrix}$$

$$S_{22} = (5)(1) + (1)(-1) + (3)(0) + (2)(1) = 5 - 1 + 0 + 2 = \boxed{6}$$

Output Feature Map (Convolution):

$$S = \begin{bmatrix} 4 & 3 \\ 2 & 6 \end{bmatrix}$$

For **cross-correlation**, use K as-is:

$$K = \begin{bmatrix} 1 & 0 \\ -1 & 1 \end{bmatrix}$$

Now compute again.

Region	Computation	Result
(1) $[1, 2; 4, 5]$	$1(1) + 2(0) + 4(-1) + 5(1) = 1 + 0 - 4 + 5$	2
(2) $[2, 0; 5, 1]$	$2(1) + 0(0) + 5(-1) + 1(1) = 2 + 0 - 5 + 1$	-2
(3) $[4, 5; 2, 3]$	$4(1) + 5(0) + 2(-1) + 3(1) = 4 + 0 - 2 + 3$	5
(4) $[5, 1; 3, 2]$	$5(1) + 1(0) + 3(-1) + 2(1) = 5 + 0 - 3 + 2$	4

b)

Output Feature Map (Cross-Correlation):

$$S_{corr} = \begin{bmatrix} 2 & -2 \\ 5 & 4 \end{bmatrix}$$

C) Practical reason kernel flipping is rarely used in CNNs

In CNNs, kernel flipping has no impact on learning, since the kernel weights are learned automatically.

Therefore, most libraries implement cross-correlation (no flipping) for computational simplicity and efficiency.